DA1 Report Content: ER Design, Relational Mapping and Normalization

Project: Satellite Imagery Knowledge Graph (GraphRAG for Earth Observation data) **Course:** BACSE202 Database Management Systems, Fall 2026–27 **Diagrams to draw:** `er_chen.png` (ER diagram) and `relational_schema.png` (relational schema)

---

1. Problem Statement

Earth-observation (EO) satellites capture thousands of images every day. Each image carries metadata:

- capture time
- footprint (location)
- cloud cover and sun elevation
- the sensor and satellite that took it
- the objects an AI model detected in it (buildings, vessels, tree cover…), each with a confidence score

In most systems this metadata is stored in separate files and tables with no links between them. So a connected question cannot be answered directly:

> "Find images showing rapid deforestation near river basins in Q3, and how trustworthy is the evidence?"

Answering it needs four things linked together:

- **what** was detected (tree cover)
- **where** (near a river basin)
- **when** (Q3)
- **how it changed** (the same forest patch shrinking between images)

2. Objectives

1. Design a database that links satellites, sensors, images, locations, river basins and detected objects.
2. Track the **same real-world object across time** (temporal lineage), so change can be measured.
3. Store **natural-language questions with their ranked evidence** and an **answer-confidence score**.
4. Raise **alerts** only for trustworthy changes, and keep an **audit trail** of edits.
5. Normalise the design to **BCNF**, so the data stays free of update anomalies.

3. Scope and Users

User	What they do
EO analyst	searches images, studies locations, asks questions, reviews alerts
Data/ML engineer	loads images and detections, registers detection models
Administrator	manages satellites, users and the audit log

In scope: optical images, detected objects, spectral indices (NDVI/NDWI), change tracking, alerts, the question/evidence log. **Out of scope:** storing the raw image pixels (only the file path is stored), and real-time downlink from satellites.

4. Functional Requirements

- **FR1** Register satellites and the sensors they carry.
- **FR2** Record each image frame with its sensor, location, time, cloud cover and file path.
- **FR3** Record the objects detected in each image: type, model, confidence, bounding box and area.
- **FR4** Store NDVI/NDWI spectral indices per image.
- **FR5** Link each location to the river basin it belongs to (optional).
- **FR6** Link two observations of the same object at different times (EVOLVED_FROM) and store the change.
- **FR7** Raise one alert per significant change and assign it to an analyst.
- **FR8** Log every question with its weights, ranked evidence and answer confidence.
- **FR9** Audit inserts, updates and deletes of detected objects.

5. Changes from the First Design (8 → 15 entities)

Change	Reason
Added **SENSOR** between SATELLITE and IMAGE_FRAME	resolution and band count belong to the instrument, not the image (this removes a transitive dependency)
Removed `satellite_id` and `resolution` from IMAGE_FRAME	both follow from `sensor_id` (image → sensor → satellite)
Removed `basin_id` from DETECTED_OBJECT	the basin follows from object → image → location → basin; storing it again was redundant
Added **DETECTION_MODEL**	replaces the free-text `model_version`; one model is used for many detections
`EVOLVED_FROM` became the associative entity **OBJECT_CHANGE**	the change has its own attributes, and ALERT must reference a change
Added **SPECTRAL_INDEX**, **APP_USER**, **ALERT**, **AUDIT_LOG**	needed for the change-detection, user and audit requirements
Separate `latitude` / `longitude` and bbox columns instead of a geometry type	Oracle XE has no PostGIS; plain numeric columns can be indexed and checked

---

6. ER Diagram

(Draw *`er_chen.png`*.)

Notation used:

- rectangle = strong entity
- double rectangle = weak entity
- rounded rectangle = associative entity
- diamond = relationship
- double diamond = identifying relationship
- double line = total participation
- 1 / N = cardinality

---

7. Entity Classification

Entity	Type	Reason
SATELLITE	Strong	own key `satellite_id`; exists independently
RIVER_BASIN	Strong	own key `basin_id`
LOCATION	Strong	own key `location_id`
IMAGE_FRAME	Strong	own key `image_id`
OBJECT_TYPE	Strong (lookup)	own key `object_type_id`
DETECTION_MODEL	Strong (lookup)	own key `model_id`
DETECTED_OBJECT	Strong, **existence-dependent**	own surrogate key `object_id`, but it cannot exist without its IMAGE_FRAME (total participation, ON DELETE CASCADE). It has no natural partial key, so it is modelled as strong.
APP_USER	Strong	own key `user_id`
QUERY_LOG	Strong	own key `query_id`
ALERT	Strong	own key `alert_id`
AUDIT_LOG	Strong (independent)	filled only by triggers; no relationships
SENSOR	**Weak**	identified by its owner SATELLITE plus the **partial key** `sensor_name`. Two satellites can each carry a sensor called "MSI". Identifying relationship: CARRIES.
SPECTRAL_INDEX	**Weak**	identified by its owner IMAGE_FRAME plus the **partial key** `index_name` (NDVI, NDWI, NBR). Identifying relationship: HAS_INDEX.
OBJECT_CHANGE	**Associative**	the recursive relationship EVOLVED_FROM (DETECTED_OBJECT → DETECTED_OBJECT) promoted to an entity, because it has attributes (change %, days, rate) and ALERT must relate to it
QUERY_EVIDENCE	**Associative**	resolves the **M:N** relationship between QUERY_LOG and DETECTED_OBJECT; carries the scores and rank

> **Implementation note (say this in the viva):** the weak entities SENSOR and SPECTRAL_INDEX are given a surrogate primary key (`sensor_id`, `index_id`) for simpler foreign keys. Their weak-entity identity is still enforced by **UNIQUE(owner_id, partial_key)**, and their existence dependency by **ON DELETE CASCADE** from the owner.

---

8. Attributes and Their Types

Key: S = simple (atomic), C = composite, MV = multi-valued, D = derived, K = key attribute, PK* = partial key. Every attribute not marked MV is single-valued.

SATELLITE

Attribute	Type
satellite_id	S, **K (PK)**
satellite_name	S, candidate key
operator_name, launch_date, orbit_type, sat_status, revisit_days	S

SENSOR (weak)

Attribute	Type
sensor_name	S, **partial key**
sensor_type, spatial_resolution_m, num_bands	S

RIVER_BASIN

Attribute	Type
basin_id	S, K (PK)
basin_code, basin_name	S, candidate keys
river_reference_point = (river_ref_lat, river_ref_lon)	**C**
area_sq_km, basin_description	S

LOCATION

Attribute	Type
location_id	S, K (PK)
location_name	S, candidate key
position = (latitude, longitude)	**C**
country, region_type	S

IMAGE_FRAME

Attribute	Type
image_id	S, K (PK)
capture_time, cloud_cover, sun_elevation, image_created_at	S
file_path	S, candidate key
image_embedding	**MV** (an ordered list of 8 numbers) and **D** (computed by an AI model from the pixels)

OBJECT_TYPE

Attribute	Type
object_type_id	S, K (PK)
type_name	S, candidate key
category, object_type_description	S

DETECTION_MODEL

Attribute	Type
model_id	S, K (PK)
(model_name, model_version)	composite candidate key
architecture, trained_on, released_date	S

DETECTED_OBJECT

Attribute	Type
object_id	S, K (PK)
bbox = (bbox_min_lat, bbox_min_lon, bbox_max_lat, bbox_max_lon)	**C**
confidence_score	S (produced by the detection model)
area_sq_m	S (measured by the model)
object_embedding	**MV, D**
attributes_json (e.g. floors, material, vessel_class, length_m)	**C / MV:** a variable set of sub-attributes per object type
object_created_at	S

OBJECT_CHANGE (associative)

Attribute	Type
change_id	S, K (PK)
change_pct	**D** (from the two objects' areas)
time_delta_days	**D** (from the two images' capture times)
rate_pct_per_month	**D** = change_pct ÷ days × 30
change_type	**D** (from change_pct thresholds)
computed_at	S

SPECTRAL_INDEX (weak)

Attribute	Type
index_name	S, **partial key**
mean_value, min_value, max_value	S

APP_USER

Attribute	Type
user_id	S, K (PK)
email	S, candidate key
full_name	S. It could be split into first/last name; it is kept whole because it is never searched by part.
user_role, user_created_at	S

QUERY_LOG

Attribute	Type
query_id	S, K (PK)
query_text, submitted_at, response_text	S
query_embedding	**MV, D**
retrieval_weights = (alpha, beta, gamma)	**C**
aggregated_confidence	**D** (from the evidence rows)

QUERY_EVIDENCE (associative)

Attribute	Type
evidence_id	S, K (PK)
semantic_score, spatial_score, temporal_score	S (computed at retrieval time)
retrieval_score	**D** = α·semantic + β·spatial + γ·temporal
rank_position	S

ALERT

Attribute	Type
alert_id	S, K (PK)
severity	**D** (from change_pct when the alert is raised)
alert_status, raised_at, resolved_at	S

AUDIT_LOG

Attribute	Type
audit_id	S, K (PK)
table_name, operation, record_id, changed_by, changed_at, old_value, new_value	S

9. Relationships

Relationship	Entities	Degree	Cardinality	Participation	Identifying?
CARRIES	SATELLITE – SENSOR	binary	1 : N	SATELLITE partial, SENSOR **total**	**Yes**
CAPTURED_BY	SENSOR – IMAGE_FRAME	binary	1 : N	SENSOR partial, IMAGE **total**	No
LOCATED_AT	LOCATION – IMAGE_FRAME	binary	1 : N	LOCATION partial, IMAGE **total**	No
BELONGS_TO	RIVER_BASIN – LOCATION	binary	1 : N	both **partial** (a city may lie outside any basin)	No
HAS_INDEX	IMAGE_FRAME – SPECTRAL_INDEX	binary	1 : N	IMAGE partial, INDEX **total**	**Yes**
DETECTED_IN	IMAGE_FRAME – DETECTED_OBJECT	binary	1 : N	IMAGE partial (an image may contain nothing), OBJECT **total**	No (existence-dependent)
HAS_TYPE	OBJECT_TYPE – DETECTED_OBJECT	binary	1 : N	TYPE partial, OBJECT **total**	No
DETECTED_BY	DETECTION_MODEL – DETECTED_OBJECT	binary	1 : N	MODEL partial, OBJECT **total**	No
EVOLVED_FROM (via OBJECT_CHANGE)	DETECTED_OBJECT – DETECTED_OBJECT	**unary (recursive)**, roles previous and current	1 : N on each role	OBJECT partial (the first sighting has no predecessor), OBJECT_CHANGE **total** to both roles	No
USED_AS_EVIDENCE / PROVIDES_EVIDENCE (via QUERY_EVIDENCE)	QUERY_LOG – DETECTED_OBJECT	binary	**M : N**	both partial; QUERY_EVIDENCE **total** to both	No

Relationship	Entities	Degree	Cardinality	Participation	Identifying?
ASKS	APP_USER – QUERY_LOG	binary	1 : N	both **partial** (anonymous questions allowed)	No
TRIGGERS	OBJECT_CHANGE – ALERT	binary	**1 : 1**	CHANGE partial (only significant ones), ALERT **total**	No
ASSIGNED_TO	APP_USER – ALERT	binary	1 : N	both **partial** (an alert may be unassigned)	No

---

10. ER-to-Relational Mapping (steps used)

1. **Strong entities** → one table each, with the key as PK (SATELLITE, RIVER_BASIN, LOCATION, IMAGE_FRAME, OBJECT_TYPE, DETECTION_MODEL, DETECTED_OBJECT, APP_USER, QUERY_LOG, ALERT, AUDIT_LOG).
2. **Weak entities** → a table containing the owner's PK as an FK, plus the partial key. The identity (owner_id, partial_key) is kept as **UNIQUE**, and a surrogate PK is added. ON DELETE CASCADE from the owner keeps the existence dependency.
   - SENSOR (UNIQUE `satellite_id, sensor_name`)
   - SPECTRAL_INDEX (UNIQUE `image_id, index_name`)
3. **1:N relationships** → FK on the N side. NOT NULL where participation is total; nullable where it is partial (`location.basin_id`, `query_log.user_id`, `alert.assigned_user_id`).
4. **1:1 relationship TRIGGERS** → FK `change_id` in ALERT (the total side) with a **UNIQUE** constraint.
5. **M:N relationship** QUERY_LOG–DETECTED_OBJECT → separate table QUERY_EVIDENCE with both FKs, UNIQUE(query_id, object_id), and the relationship's own attributes.
6. **Recursive relationship with attributes** EVOLVED_FROM → separate table OBJECT_CHANGE with two FKs to DETECTED_OBJECT (`prev_object_id`, `curr_object_id`), UNIQUE on the pair, CHECK (prev ≠ curr).
7. **Composite attributes** → flattened into their components (latitude/longitude; bbox_min_lat … bbox_max_lon).
8. **Multi-valued attributes** (embeddings, attributes_json) → stored as a single JSON value with an `IS JSON` check. This is justified in §13.

11. Relational Schema

(Draw `relational_schema.png`.) PKs are underlined; FKs are shown with arrows.

```
SATELLITE      (satellite_id PK, satellite_name UK, operator_name, launch_date,
orbit_type, sat_status, revisit_days)
SENSOR         (sensor_id PK, satellite_id FK→SATELLITE, sensor_name, sensor_type,
spatial_resolution_m, num_bands)
                UNIQUE(satellite_id, sensor_name)
RIVER_BASIN    (basin_id PK, basin_name UK, basin_code UK, river_ref_lat, river_ref_lon,
area_sq_km, basin_description)
LOCATION       (location_id PK, basin_id FK→RIVER_BASIN NULL, location_name UK, country,
latitude, longitude, region_type)
IMAGE_FRAME    (image_id PK, sensor_id FK→SENSOR, location_id FK→LOCATION, capture_time,
cloud_cover, sun_elevation,
                file_path UK, image_embedding, image_created_at)       UNIQUE(sensor_id,
location_id, capture_time)
OBJECT_TYPE    (object_type_id PK, type_name UK, category, object_type_description)
DETECTION_MODEL(model_id PK, model_name, model_version, architecture, trained_on,
released_date)
                UNIQUE(model_name, model_version)
DETECTED_OBJECT(object_id PK, image_id FK→IMAGE_FRAME, object_type_id FK→OBJECT_TYPE,
model_id FK→DETECTION_MODEL,
                confidence_score, bbox_min_lat, bbox_min_lon, bbox_max_lat, bbox_max_lon,
area_sq_m,
                object_embedding, attributes_json, object_created_at)
OBJECT_CHANGE  (change_id PK, prev_object_id FK→DETECTED_OBJECT, curr_object_id
FK→DETECTED_OBJECT, change_type,
                change_pct, time_delta_days, rate_pct_per_month, computed_at)
UNIQUE(prev_object_id, curr_object_id)
SPECTRAL_INDEX (index_id PK, image_id FK→IMAGE_FRAME, index_name, mean_value, min_value,
max_value)
                UNIQUE(image_id, index_name)
APP_USER       (user_id PK, full_name, email UK, user_role, user_created_at)
QUERY_LOG      (query_id PK, user_id FK→APP_USER NULL, query_text, query_embedding,
submitted_at, response_text,
                retrieval_weights_json, aggregated_confidence)
QUERY_EVIDENCE (evidence_id PK, query_id FK→QUERY_LOG, object_id FK→DETECTED_OBJECT,
semantic_score, spatial_score,
                temporal_score, retrieval_score, rank_position)        UNIQUE(query_id,
object_id)
ALERT          (alert_id PK, change_id FK→OBJECT_CHANGE UK, assigned_user_id FK→APP_USER
NULL, severity,
                alert_status, raised_at, resolved_at)
AUDIT_LOG      (audit_id PK, table_name, operation, record_id, changed_by, changed_at,
old_value, new_value)
```

12. Keys and Integrity Constraints

12.1 Key summary

Table	Primary key	Candidate / unique keys	Foreign keys	Partial key (weak entity)
SATELLITE	satellite_id	satellite_name	none	none
SENSOR	sensor_id	(satellite_id, sensor_name)	satellite_id	**sensor_name**
RIVER_BASIN	basin_id	basin_code; basin_name	none	none
LOCATION	location_id	location_name	basin_id (nullable)	none
IMAGE_FRAME	image_id	file_path; (sensor_id, location_id, capture_time)	sensor_id, location_id	none
OBJECT_TYPE	object_type_id	type_name	none	none
DETECTION_MODEL	model_id	(model_name, model_version)	none	none
DETECTED_OBJECT	object_id	none	image_id, object_type_id, model_id	none (surrogate)
OBJECT_CHANGE	change_id	(prev_object_id, curr_object_id)	prev_object_id, curr_object_id	none
SPECTRAL_INDEX	index_id	(image_id, index_name)	image_id	**index_name**
APP_USER	user_id	email	none	none
QUERY_LOG	query_id	none	user_id (nullable)	none
QUERY_EVIDENCE	evidence_id	(query_id, object_id)	query_id, object_id	none
ALERT	alert_id	change_id	change_id, assigned_user_id (nullable)	none

Table	Primary key	Candidate / unique keys	Foreign keys	Partial key (weak entity)
AUDIT_LOG	audit_id	none	none (record_id is a generic reference)	none

12.2 Integrity constraints

- **Domain (CHECK):**
  - `cloud_cover` 0–100
  - `sun_elevation` 0–90
  - `confidence_score` 0–1
  - `area_sq_m` > 0
  - latitude −90…90 and longitude −180…180
  - `min ≤ mean ≤ max` and all within −1…1 for spectral indices
  - all evidence scores 0–1
  - `rank_position` > 0
  - `time_delta_days` > 0
- **Enumerations (CHECK … IN):**
  - orbit_type, sat_status, sensor_type, region_type, index_name
  - change_type, severity, alert_status, user_role, audit operation
- **Structural (CHECK):**
  - bbox min < max
  - `prev_object_id ≠ curr_object_id`
  - `resolved_at` is set only when the status is RESOLVED
  - JSON columns must hold valid JSON (`IS JSON`)
- **Entity integrity:** every PK is NOT NULL and unique.
- **Referential integrity:**
  - CASCADE where the child cannot exist alone (sensor, object, index, change, evidence, alert)
  - SET NULL where the link is optional (location.basin_id, query_log.user_id, alert.assigned_user_id)
  - RESTRICT (default) where deleting would lose history (deleting a sensor that still has images is blocked)
- **Rules enforced by triggers** (they need other rows, so CHECK cannot express them):
  - a capture time cannot be in the future

- an EVOLVED_FROM pair must be at the same location, and the previous object must be older

---

13. Functional Dependencies and Normalization

13.1 Starting point: the unnormalised relation (UNF)

The data an EO pipeline produces, as one flat record per image with repeating groups:

```
EO_RECORD( file_path, capture_time, cloud_cover, sun_elevation,
           satellite_name, operator_name, launch_date, orbit_type, sat_status,
revisit_days,
           sensor_name, sensor_type, spatial_resolution_m, num_bands,
           location_name, country, latitude, longitude, region_type,
           basin_code, basin_name, river_ref_lat, river_ref_lon, area_sq_km,
           { object_no, type_name, category, model_name, model_version, architecture,
             confidence, bbox, area_sq_m, attributes }*,          ← repeating group 1
           { index_name, mean_value, min_value, max_value }* )    ← repeating group 2
```

Problems:

- repeating groups (many objects and many indices per image)
- a non-atomic `bbox`
- massive redundancy: Sentinel-2A's operator and orbit are repeated on every image, and the Amazon basin details on every Manaus image

Example anomalies:

- Update: changing a basin's area must be repeated in every row of every image in that basin.
- Insert: a new satellite cannot be recorded until it captures an image.
- Delete: deleting the last image of a location also deletes the location's country and coordinates.

13.2 Functional dependencies (FDs)

#	FD
FD1	satellite_name → operator_name, launch_date, orbit_type, sat_status, revisit_days
FD2	(satellite_name, sensor_name) → sensor_type, spatial_resolution_m, num_bands
FD3	file_path → capture_time, cloud_cover, sun_elevation, satellite_name, sensor_name, location_name
FD4	(satellite_name, sensor_name, location_name, capture_time) → file_path (an instrument photographs a place only once at a given instant)
FD5	location_name → country, latitude, longitude, region_type, basin_code
FD6	basin_code → basin_name, river_ref_lat, river_ref_lon, area_sq_km; basin_name → basin_code
FD7	type_name → category, object_type_description
FD8	(model_name, model_version) → architecture, trained_on, released_date
FD9	(file_path, object_no) → type_name, model_name, model_version, confidence, bbox_min_lat, bbox_min_lon, bbox_max_lat, bbox_max_lon, area_sq_m, attributes
FD10	(file_path, index_name) → mean_value, min_value, max_value

13.3 First Normal Form (1NF)

Rule: atomic values only, no repeating groups.

- `bbox` is split into 4 atomic columns.
- The repeating groups are moved out, giving one row per detected object and one row per spectral index:

```
R1_OBJECT( file_path, object_no, capture_time, cloud_cover, sun_elevation,
           satellite_name, operator_name, launch_date, orbit_type, sat_status,
revisit_days,
           sensor_name, sensor_type, spatial_resolution_m, num_bands,
           location_name, country, latitude, longitude, region_type,
           basin_code, basin_name, river_ref_lat, river_ref_lon, area_sq_km,
           type_name, category, model_name, model_version, architecture,
           confidence, bbox_min_lat, bbox_min_lon, bbox_max_lat, bbox_max_lon, area_sq_m,
attributes )
           Key = (file_path, object_no)

R1_INDEX(  file_path, index_name, mean_value, min_value, max_value, …same image columns…
)
           Key = (file_path, index_name)
```

13.4 Second Normal Form (2NF)

Rule: 1NF, and no non-key attribute depends on only part of a composite key.

- In R1_OBJECT, the key is (file_path, object_no). All the image, satellite, sensor, location and basin columns depend only on **file_path** (FD3 and everything reached from it). That is a **partial dependency**.
- The same happens in R1_INDEX: the image columns depend only on file_path.

Decomposition:

```
R2_IMAGE  ( file_path PK, capture_time, cloud_cover, sun_elevation,
            satellite_name, operator_name, launch_date, orbit_type, sat_status,
revisit_days,
            sensor_name, sensor_type, spatial_resolution_m, num_bands,
            location_name, country, latitude, longitude, region_type,
            basin_code, basin_name, river_ref_lat, river_ref_lon, area_sq_km )
R2_OBJECT ( file_path FK, object_no, type_name, category, model_name, model_version,
architecture,
            confidence, bbox…, area_sq_m, attributes )             Key = (file_path,
object_no)
R2_INDEX  ( file_path FK, index_name, mean_value, min_value, max_value )   Key =
(file_path, index_name)
```

R2_INDEX is already in BCNF.

13.5 Third Normal Form (3NF)

Rule: 2NF, and no **transitive** dependency (non-key → non-key).

Transitive dependencies in R2_IMAGE:

- file_path → satellite_name → operator_name, launch_date, … (FD1)
- file_path → (satellite_name, sensor_name) → spatial_resolution_m, … (FD2)
- file_path → location_name → country, latitude, … (FD5) → basin_code → basin_name, … (FD6)

Transitive dependencies in R2_OBJECT:

- (file_path, object_no) → type_name → category (FD7)
- (file_path, object_no) → (model_name, model_version) → architecture (FD8)

Decomposition:

```
SATELLITE       ( satellite_name PK, operator_name, launch_date, orbit_type, sat_status,
revisit_days )
SENSOR          ( satellite_name FK, sensor_name, sensor_type, spatial_resolution_m,
num_bands )   Key = (satellite_name, sensor_name)
RIVER_BASIN     ( basin_code PK, basin_name, river_ref_lat, river_ref_lon, area_sq_km )
LOCATION        ( location_name PK, country, latitude, longitude, region_type, basin_code
FK )
IMAGE_FRAME     ( file_path PK, satellite_name, sensor_name  FK→SENSOR, location_name FK,
capture_time, cloud_cover, sun_elevation )
OBJECT_TYPE     ( type_name PK, category, object_type_description )
DETECTION_MODEL ( model_name, model_version  PK, architecture, trained_on, released_date
)
DETECTED_OBJECT ( file_path FK, object_no, type_name FK, model_name, model_version FK,
confidence, bbox…, area_sq_m, attributes )
SPECTRAL_INDEX  ( file_path FK, index_name, mean_value, min_value, max_value )
```

> Note: **satellite_name is not stored on IMAGE_FRAME on its own.** It is only part of the composite FK to SENSOR. This is exactly why the first design's `IMAGE_FRAME.satellite_id` and `resolution` were removed. The same reasoning removed `DETECTED_OBJECT.basin_id`: object → image → location → basin is a transitive chain.

13.6 Boyce-Codd Normal Form (BCNF)

Rule: for every non-trivial FD X → Y, X must be a **superkey**.

Relation	Determinants	All superkeys?
SATELLITE	satellite_name	✓
SENSOR	(satellite_name, sensor_name)	✓
RIVER_BASIN	basin_code; basin_name (candidate key, FD6)	✓ both are candidate keys
LOCATION	location_name	✓
IMAGE_FRAME	file_path; (satellite_name, sensor_name, location_name, capture_time) (FD4)	✓ both are candidate keys (overlapping keys, still BCNF)
OBJECT_TYPE	type_name	✓
DETECTION_MODEL	(model_name, model_version)	✓
DETECTED_OBJECT	(file_path, object_no)	✓
SPECTRAL_INDEX	(file_path, index_name)	✓

All nine relations are in **BCNF**. The decomposition is **lossless**: every split shares a key, joined through a foreign key. It is also **dependency-preserving**: each FD1–FD10 lies inside one relation.

13.7 Surrogate keys (final step)

Long natural keys (file_path, composite sensor and model keys) are replaced by **numeric surrogate PKs** (`image_id`, `sensor_id`, `model_id`, …). The natural keys are kept as **UNIQUE** constraints, so no FD is lost. The relations stay in BCNF, because each natural key is still a candidate key, and the FKs become short integers.

13.8 The application tables (from the requirements, not the pipeline record)

Relation	FDs	Normal form
APP_USER	user_id → all; email → all	BCNF
QUERY_LOG	query_id → all	BCNF
QUERY_EVIDENCE	evidence_id → all; (query_id, object_id) → all	BCNF
ALERT	alert_id → all; change_id → all	BCNF
AUDIT_LOG	audit_id → all	BCNF
OBJECT_CHANGE	change_id → all; (prev, curr) → all; **change_pct → change_type**; **(change_pct, time_delta_days) → rate_pct_per_month**	**3NF/BCNF with deliberately stored derived values**, see §13.9

13.9 Deliberate denormalization (justified)

Stored value	Could be derived from	Why it is stored anyway	How anomalies are prevented
OBJECT_CHANGE.change_pct, time_delta_days, rate_pct_per_month, change_type	the two objects' areas and their images' capture times	a change is a **historical measurement**. Alerts and reports must show the value that triggered them, and recomputing joins for every report is slow	written **only** by `pkg_eo.compute_changes` (via MERGE), using `fn_classify_change` for change_type, so the values cannot drift. Strictly, change_pct → change_type is a dependency between non-key attributes; it is kept as a controlled derived column
ALERT.severity	change_pct	records the severity **at the time the alert was raised**	set only by `pkg_eo.raise_alerts`
QUERY_EVIDENCE.retrieval_score, rank_position	the three scores + the weights in QUERY_LOG	a **snapshot of one retrieval run**; tuning the weights later must not rewrite old answers	written once per question, in one transaction
QUERY_LOG.aggregated_confidence	the evidence rows	shown on every result page	written only by `pkg_eo.finalize_query`
image_embedding, object_embedding, query_embedding, attributes_json (JSON, i.e. multi-valued)	none	an embedding is always read and written **as a whole vector** (it is never queried by element), so a child table with 8 rows per object would add joins without benefit. `attributes_json` holds different keys per object type,	`CHECK (… IS JSON)` guarantees validity. In the Neo4j extension (DA3) the same vectors become native vector-index properties

Stored value	Could be derived from	Why it is stored anyway	How anomalies are prevented
		which would otherwise mean many mostly-NULL columns	

13.10 Final result

15 relations, all in **BCNF** except for the documented derived columns. All have a surrogate PK, the natural keys are kept as UNIQUE constraints, and every FK is indexed. These 15 relations are exactly the tables created in `database/schema.sql`.